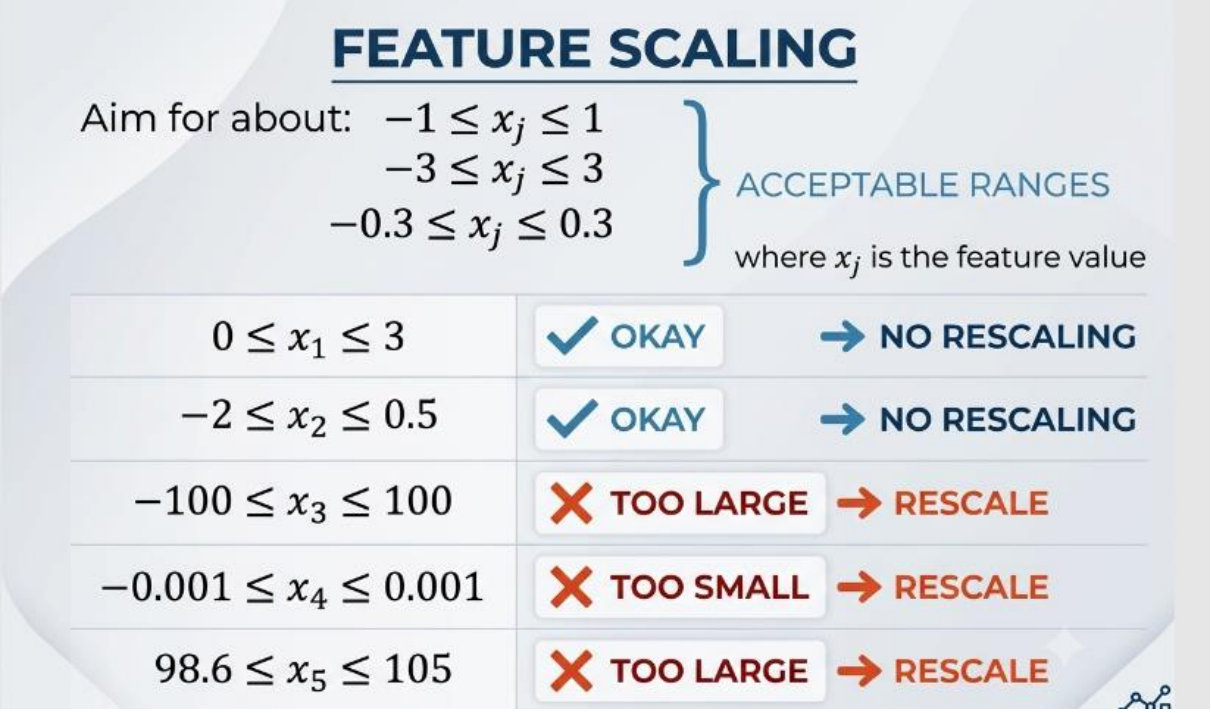
Feature scaling

Feature scaling is a data preprocessing technique used in machine learning to normalize the range of independent variables or features in a dataset. This process ensures that all features contribute equally to the model, preventing variables with large magnitudes from dominating those with smaller ranges. By adjusting data to a common scale, such as [-1, 1] or [0, 1], the performance and convergence speed of optimization algorithms, such as gradient descent, are significantly improved.



1. The Prediction Formula

The core equation for predicting house price using two features is:

price = $w_1 \cdot x_1 + w_2 \cdot x_2 + b$

Where:

- w_1 = weight (parameter) for feature x_1 (size in sq ft)
- w_2 = weight (parameter) for feature x_2 (number of bedrooms)
- b = bias / intercept term
- price = predicted output (in $k)

2. Feature Definitions and Ranges

Two input features are used in this model:

Feature	Symbol	Description	Range
Size	x_1	House area in square feet	300 – 2,000 ft² → LARGE range
Bedrooms	x_2	Number of bedrooms	0 – 5 → SMALL range

Key insight: x_1 has a MUCH larger numerical range than x_2. This means the weights w_1 and w_2 will have very different magnitudes to produce comparable contributions to the price prediction.

3.Training Example

A single training example is given:

x_1 = 2,000 ft², x_2 = 5 bedrooms, price = $500k

This real house (with a known price of $500,000) is used to evaluate whether a given set of parameters (w_1, w_2, b) produces a reasonable prediction.

4. Parameter Scale Matters — Two Scenarios Compared

The key question is: how big should w_1 and w_2 be relative to each other?

Two different parameter sets are compared below:

✗ Scenario A — Wrong Scale	✓✓ Scenario B — Correct Scale
w_1 = 50, w_2 = 0.1, b = 50 **Calculation:** = 50 × 2,000 + 0.1 × 5 + 50 = 100,000k + 0.5k + 50k **= $100,050.5k = $100,050,500** 🚨 Wildly overestimated ! 200× too high.	w_1 = 0.1, w_2 = 50, b = 50 **Calculation:** = 0.1 × 2,000 + 50 × 5 + 50 = 200k + 250k + 50k **= $500k ✓**

	✓Matches the actual price. More reasonable.

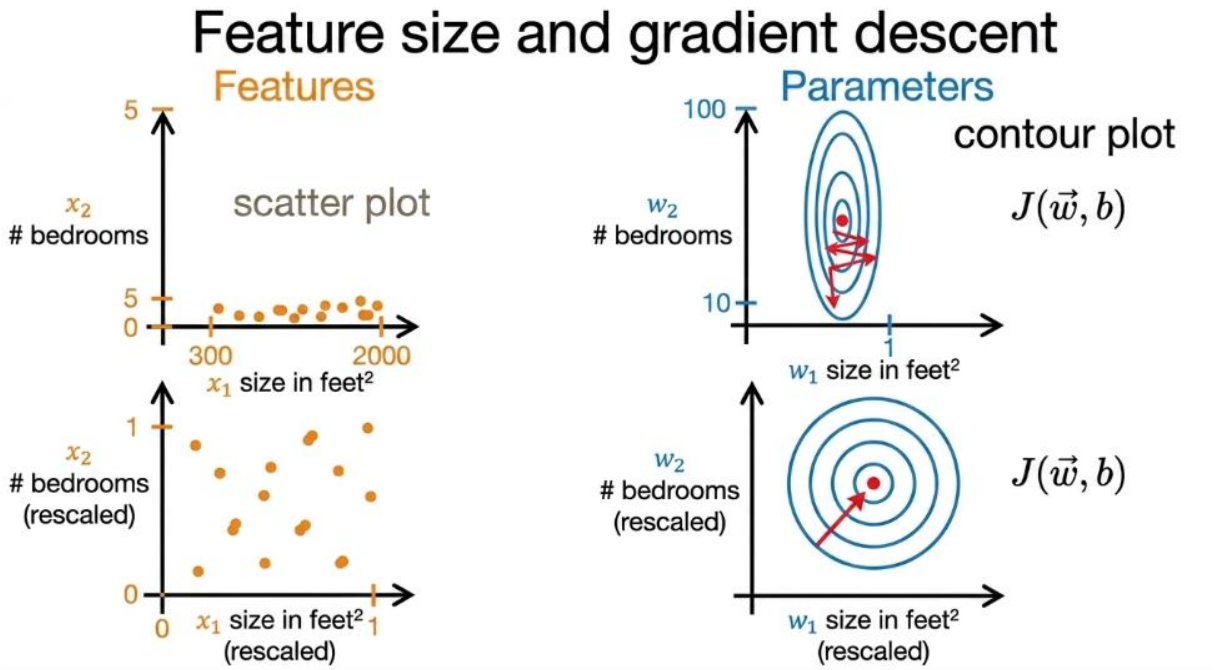
5. Key Takeaway : Relationship Between Feature Scale and Parameter Size

When features have very different numerical ranges, the parameters that multiply them must compensate:

• A LARGE-range feature (x_1 = size, up to 2,000) needs a SMALL weight → $w_1 \approx 0.1$

• A SMALL-range feature (x_2 = bedrooms, up to 5) needs a LARGE weight → $w_2 \approx 50$

This is why feature scaling (e.g. normalization or standardization) is important in practice, it allows all features to have similar ranges and simplifies parameter tuning during gradient descent



Feature scaling is an essential preprocessing step in machine learning that ensures all features contribute equally to the model, preventing gradient descent from zigzagging and allowing it to converge faster toward the global minimum.

Several techniques exist to achieve proper feature scaling, each with its own advantages depending on the data distribution and the presence of outliers, the most widely used being Min-Max Normalization, Mean Normalization, and Z-score Standardization.

1. Min-Max Normalization (Rescaling)

Formula:

$x' = (x - \min) / (\max - \min)$

Scales all values to the range [0, 1]. The minimum becomes 0, the maximum becomes 1, everything else falls in between.

When to use: when you need values strictly between 0 and 1, e.g. for neural networks with sigmoid activation, image pixel values.

Drawback: very sensitive to outliers, one extreme value compresses all others into a tiny range.

2. Mean Normalization

Formula:

$x' = (x - \mu) / (\max - \min)$

Similar to min-max but subtracts the mean instead of the minimum, so the result is centered around 0, typically in [−1, 1].

When to use: when you want centered data but don't need to assume a Gaussian distribution.

3. Z-Score Standardization (Standardization)

Formula:

$x' = (x - \mu) / \sigma$

Subtracts the mean and divides by the standard deviation. The result has mean = 0 and std = 1.

The standard deviation (σ) measures how spread out the values are around the mean.

Formula

$$\boldsymbol{\sigma = \sqrt{[\ (1/n) \times \Sigma(x_i - \mu)^2\ ]}}$$

Step by step:

1. Calculate the mean μ
2. Subtract the mean from each value → $(x_i - \mu)$
3. Square each difference → $(x_i - \mu)^2$

4. Average all squared differences → divide by n
5. Take the square root → √

Simple Example

Values: 2, 4, 4, 6, 4

- μ = (2+4+4+6+4) / 5 = 4
- Differences: (−2, 0, 0, 2, 0)
- Squared: (4, 0, 0, 4, 0)
- Average: (4+0+0+4+0) / 5 = 1.6
- σ = √1.6 = ≈ 1.26

What it tells you

σ value	**Meaning**
Small σ	Values are clustered tightly around the mean
Large σ	Values are widely spread from the mean
σ = 0	All values are identical

In the house price example from the dataset (house price), $\sigma_1 = 450$ for house size, meaning most houses are within 450 ft² of the average (600 ft²), which reflects a fairly wide spread across the dataset.

When to use: the most widely used method in ML. Works well with algorithms that assume normally distributed data: linear regression, logistic regression, SVM, PCA, neural networks.

Advantage over min-max: robust to outliers because it uses spread (σ) rather than the fixed min/max.

4. Max Absolute Scaling

Formula:

x' = x / |max(x)|

Divides by the largest absolute value. Result is in **[−1, 1]** and preserves zero values (does not shift the data).

When to use: sparse data (lots of zeros), e.g. text/NLP features. You don't want to destroy the sparsity by adding a shift.

Top-left scatter plot (original data)

The original data is plotted with:

- x-axis: house size (300 to 2,000 ft²)
- y-axis: number of bedrooms (0 to 5)
- $\mu_1 = 600$ → the average house size across all training examples
- $\mu_2 = 2.3$ → the average number of bedrooms

The data is not centered — both axes start far from zero, and the ranges are very different from each other.

The Mean Normalization formulas

For x_1 (size):

$x_1 = (x_1 - \mu_1) / (\max - \min) = (x_1 - 600) / (2000 - 300)$

You subtract the mean (600) to center the data, then divide by the full range (1700) to scale it.

For x_2 (bedrooms):

$x_2 = (x_2 - \mu_2) / (\max - \min) = (x_2 - 2.3) / (5 - 0)$

Same idea: subtract the mean (2.3), divide by the range (5).

Resulting normalized ranges

Feature	**Original range**	**After normalization**
x_1 (size)	300 – 2,000	$-0.18 \le x_1 \le 0.82$
x_2 (bedrooms)	0 – 5	$-0.46 \le x_2 \le 0.54$

Both features now live roughly in the range [−1, 1], centered near 0.

Note that the range isn't perfectly symmetric (e.g. −0.18 to 0.82 for x_1) because the mean (600) is not exactly in the middle of [300, 2000].

Bottom-left scatter plot (normalized data)

After applying the formulas, the same data points are re-plotted. Now:

- Both axes go from about −1 to 1
- The data is centered around (0, 0)
- The two features have comparable scales

This symmetrical, centered distribution is what makes gradient descent work much more efficiently, instead of bouncing across a very elongated cost surface, it converges smoothly.

Why not just use min-max scaling?

Min-max scaling (dividing only by max−min without subtracting the mean) shifts values to [0, 1] but doesn't center them around zero. Mean normalization ensures the **mean of each feature becomes 0**, which is often preferred for algorithms sensitive to feature distribution.

Z-score Normalization

The Formula

$x' = (x - \mu) / \sigma$

Two things happen:

subtract μ → centers the data around 0

divide by σ → scales by how spread out the data is

The Values

Feature x_1 (house size):

- Range: $300 \leq x_1 \leq 2000$
- Mean: $\mu_1 = 600$
- Standard deviation: $\sigma_1 = 450$

Feature x_2 (bedrooms):

- Range: $0 \leq x_2 \leq 5$
- Mean: $\mu_2 = 2.3$
- Standard deviation: $\sigma_2 = 1.4$

Step-by-Step Calculation

For x_1 — applying the formula at the boundaries:

At $x_1 = 300$ (minimum):

$(300 - 600) / 450 = -300 / 450 =$ **-0.67**

At $x_1 = 2000$ (maximum):

$(2000 - 600) / 450 = 1400 / 450 =$ **≈ 3.1**

So after normalization: **$-0.67 \leq x_1 \leq 3.$**

For x_2 — applying the formula at the boundaries:

At $x_2 = 0$ (minimum):

$(0 - 2.3) / 1.4 = -2.3 / 1.4 =$ **≈ -1.6**

At $x_2 = 5$ (maximum):

$(5 - 2.3) / 1.4 = 2.7 / 1.4 =$ **≈ 1.9**

So after normalization: **$-1.6 \leq x_2 \leq 1.9$**

What the Bell Curve Represents

The σ illustrated on the slide is the standard deviation of x_1. It tells you how far the values typically spread from the mean. A large σ (450 for size) means the data is widely spread — so dividing by it brings everything closer together.

Comparison: Z-score vs Mean Normalization

Notice that x_1 after z-score goes up to **3.1**, not bounded to [−1, 1] like mean normalization — because z-score uses σ, not the fixed range. Values beyond ±3 are statistically rare (outliers).

Key Intuition

Z-score tells you **how many standard deviations** a value is from the mean:

- $x_1 = 2000 \rightarrow$ 3.1σ above the mean → unusually large house
- $x_1 = 300 \rightarrow$ 0.67σ below the mean → slightly smaller than average
- $x_1 = 600 \rightarrow 0\sigma \rightarrow$ exactly average → becomes **0** after normalization